1 Lesson 14-10-26

Last week we built a small kernel: unprivileged code calls the kernel through system calls (`svc`), and only the kernel touches the hardware. This week's starting code runs two tasks, A and then B, both unprivileged, each with its own stack. But every byte of RAM is still shared: the kernel's variables, task A's and task B's sit side by side, and any of them can write anywhere.

1.1 Memory

In this lesson we start covering memory management. Memory is one shared resource, and several programs want it at once. The OS has four jobs:

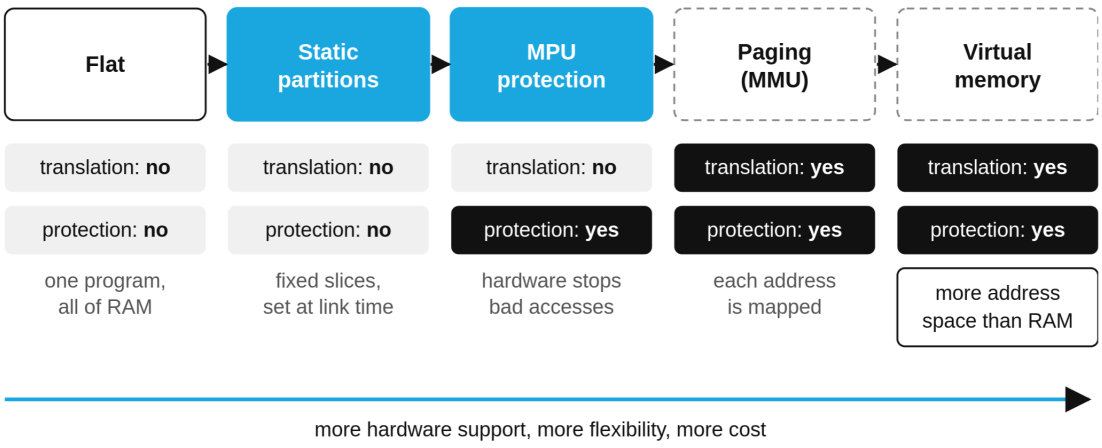
- **Placement**: decide where each program's code, data and stack go, so they do not overlap;
- **Sharing**: let programs use the same memory when they should (one copy of a library, a buffer passed to a device);
- **Protection**: stop a program from reading or writing memory that is not its own, by mistake or on purpose;
- **More programs than memory**: when the programs together need more memory than there is, keep only the parts in use in RAM.

There are a number of mechanisms used to perform memory management and protection.

1.1.1 Translation and protection

First, we should distinguish between **translation** and **protection**.

- **Translation** means the address the program uses is changed into another address before it reaches memory;
- **Protection** means the hardware stops accesses that are not allowed.



We now present an evolution timeline of memory management techniques.

- **Flat**: one program, or a kernel and its tasks together, share all of RAM. No translation, no protection;
- **Static partitions**: each task gets a fixed slice of RAM, set at link time. Still no protection: nothing stops a task from writing outside its slice;
- **MPU protection**: a hardware unit checks every access against a list of allowed regions and stops bad ones. Protection, but no translation: an address means the same thing for everyone;
- **Paging (MMU)**: hardware maps each address the program uses to a different physical place. Translation and protection;
- **Virtual memory**: paging plus the disk, so programs can use more memory than the RAM has.

Note that each step needs more hardware and buys more flexibility.